# Proximal Policy Optimization vs Deep Q-Network vs Advantage Actor-Critic: A Comparative Study on Pong

**Minh Tri An** man29@uwo.ca
*Department of Computer Science*
*University of Western Ontario*
*London, Canada*
**Samuel Joseph Dietrich** sdietr2@uwo.ca
*Department of Computer Science*
*University of Western Ontario*
*London, Canada*
**Raghav Gulati** rgulati7@uwo.ca
*Department of Computer Science*
*University of Western Ontario*
*London, Canada*
**Cherie Duong** cduong32@uwo.ca
*Department of Computer Science*
*University of Western Ontario*
*London, Canada*

## Abstract

Reinforcement learning (RL) has produced remarkable results in game-playing environments, yet practitioners frequently struggle to select among the many available algorithms. In this work we conduct a rigorous empirical comparison of three foundational deep RL algorithms—Proximal Policy Optimization (PPO), Deep Q-Network (DQN), and Advantage Actor-Critic (A2C)—on the Atari Pong environment using the Gymnasium API. We train each algorithm under identical hardware and preprocessing conditions and evaluate them on three axes: final game score, sample efficiency (time steps to reach a positive score), and computational throughput (frames per second). Our results show that PPO achieves the best trade-off between speed and stability, breaking into positive scores earliest (at ≈1.0M steps) and converging to a near-perfect final score of ≈20. DQN requires a longer burn-in period—still at $\bar{R} \approx -6.57$ at 2.5M steps—before ultimately reaching a comparable final score (≈20) at ≈3.5M steps, and is further constrained by the memory overhead of its experience replay buffer. A2C converges the slowest among the policy-gradient methods (positive rewards at ≈1.2M steps) and attains a near-perfect final score of ≈20, yet sustains the highest processing throughput throughout training (≈950 FPS). These findings offer concrete guidance for algorithm selection under different resource and performance constraints.

**Keywords:** Reinforcement Learning, PPO, DQN, A2C, Atari Pong

## 1 Introduction

Deep reinforcement learning (deep RL) has emerged as a powerful paradigm for sequential decision-making, achieving superhuman performance across many benchmark tasks. Despite this progress, the landscape of algorithms remains complex, and their relative strengths are highly context-dependent. Practitioners, especially those working with limited computational resources, must decide which algorithm converges fastest, achieves the best asymptotic performance, and uses memory most efficiently. No single deep RL algorithm dominates across all dimensions. Value-based methods such as DQN excel in sample efficiency through experience replay but incur significant memory overhead and can be sensitive to hyperparameters Mnih et al. (2013). Policy-gradient methods such as A2C are on-policy and parallelizable, yet often suffer from high variance and slower convergence Mnih et al. (2016). PPO was introduced to improve stability while retaining the simplicity of policy gradients Schulman et al. (2017), but its advantages over DQN and A2C remain context-dependent. Existing comparative studies often evaluate algorithms across many Atari games or under inconsistent preprocessing and hardware setups, making fair comparison difficult and limiting the ability to attribute performance differences to the algorithms themselves Henderson et al. (2019); Fuente and Vidal Guerra (2024). We present a controlled comparison of PPO, DQN, and A2C on the Atari Pong environment under identical preprocessing, vectorization, and hardware conditions. By minimizing confounding factors, we isolate differences in sample efficiency, final performance, and computational cost. Our study complements large-scale benchmarks by providing focused insight into the behavior of each algorithm in a standardized setting.

## 2 Related Work

Recent work has explored empirical comparisons between major deep reinforcement learning algorithms to better understand their relative strengths and trade-offs. Fuente and Vidal Guerra (2024) conducted a comparative study of DQN, PPO, and A2C, analyzing their performance across multiple environments. Their results highlight that no single algorithm consistently outperforms the others; instead, performance depends heavily on factors such as environment complexity, training stability, and computational resources. However, many comparative studies evaluate algorithms across a broad suite of environments, making it difficult to isolate the impact of specific design choices. In addition, differences in preprocessing pipelines, hyperparameters, and hardware configurations can introduce confounding variables that limit the fairness of comparisons. These issues have been noted in prior work Henderson et al. (2019), which emphasizes the importance of standardized evaluation protocols in reinforcement learning.

Deep Q-Networks (DQN), introduced by Mnih et al. (2013), combine Q-learning with deep neural networks, experience replay, and a target network to stabilize training. DQN achieved human-level performance on Atari benchmarks and remains a strong baseline for discrete control tasks. Asynchronous Advantage Actor-Critic (A3C), proposed by Mnih et al. (2016), uses multiple parallel workers to decorrelate training data. A2C is its synchronous variant, offering improved stability and compatibility with modern hardware while maintaining the benefits of actor-critic methods. Proximal Policy Optimization (PPO), in-

troduced by Schulman et al. (2017), improves the stability of policy gradient methods by constraining policy updates using a clipped objective function. PPO has become widely adopted due to its balance between simplicity and strong empirical performance.

## 3 Methodology

### 3.1 Environment

We evaluated all reinforcement learning agents on the Atari Pong environment provided by the Gymnasium framework Towers et al. (2025). The environment follows a standard Markov Decision Process (MDP) formulation where the agent interacts with a simulated Pong game by selecting discrete actions at each timestep.

The observation space consists of raw RGB frames from the game screen, which are preprocessed by converting to grayscale, resizing, and stacking consecutive frames to capture temporal dynamics Krebsbach (2016).

The action space is discrete and includes actions such as moving the paddle up, moving it down, or taking no action.

The reward function is sparse: the agent receives +1 for scoring a point, -1 for losing a point, and 0 otherwise. This sparse reward structure makes the task challenging and requires efficient exploration strategies.

**Game Objective:** The game ends when one agent reaches 21 points. The final reward range for an episode is therefore approximately $[-21, +21]$.

### 3.2 Models

We compare three reinforcement learning algorithms: Proximal Policy Optimization (PPO), Deep Q-Network (DQN), and Advantage Actor-Critic (A2C). These methods represent different paradigms in reinforcement learning, namely policy-based, value-based, and hybrid approaches. All models use deep neural networks to approximate either value functions, policies, or both, enabling them to operate directly on high-dimensional visual inputs from the environment.

#### 3.2.1 Proximal Policy Optimization (PPO)

Proximal Policy Optimization (PPO) is a policy-based reinforcement learning method designed to improve training stability while maintaining efficient policy updates. Instead of directly applying large gradient updates that may destabilize learning, PPO introduces a constraint that limits how much the new policy can deviate from the old policy.

This is achieved by optimizing a clipped surrogate objective, which ensures that probability ratios between the new and old policies remain within a controlled range. As a result, the policy is updated gradually, preventing destructive updates while still allowing meaningful improvement over time.

Intuitively, PPO can be understood as an agent that cautiously improves its decision-making strategy by only accepting changes that improve expected reward without significantly altering previously learned behaviors. This makes PPO particularly robust in complex environments with noisy or sparse rewards.

Following prior work Schulman et al. (2017), PPO has become one of the most widely used reinforcement learning algorithms due to its strong empirical performance and stability.

#### 3.2.2 Deep Q-Network (DQN)

Deep Q-Network (DQN) is a value-based reinforcement learning algorithm that approximates the optimal action-value function $Q(s, a)$ using a deep neural network. The goal of DQN is to learn the expected cumulative reward for taking a given action in a given state and following the optimal policy thereafter.

DQN is trained using off-policy learning, where experiences collected by the agent are stored in an experience replay buffer. During training, mini-batches are randomly sampled from this buffer, which breaks temporal correlations between consecutive samples and improves training stability and efficiency.

In addition, DQN uses a separate target network that is periodically updated with the weights of the main network. This technique helps stabilize Q-value targets and reduces oscillations during training.

DQN is particularly well-suited for environments with discrete action spaces, such as Atari games, and has demonstrated strong performance in benchmark tasks Mnih et al. (2013).

#### 3.2.3 Advantage Actor-Critic (A2C)

Advantage Actor-Critic (A2C) is a hybrid reinforcement learning method that combines policy-based and value-based learning. It consists of two components: an actor, which selects actions according to a learned policy, and a critic, which evaluates the quality of those actions by estimating the value of states.

The critic computes a value estimate $V(s)$, which is used to calculate the advantage function. The advantage measures how much better or worse an action performed compared to the expected value. This signal is used to update the actor by increasing the probability of actions with positive advantage and decreasing the probability of actions with negative advantage.

By using the advantage function, A2C reduces its variance in policy gradient updates, leading to more stable and efficient learning compared to the naive policy gradient methods. Unlike asynchronous variants such as A3C, A2C performs synchronous updates, making it easier to implement and more suitable for modern GPU-based training pipelines Mnih et al. (2016).

### 3.3 Evaluation Metrics

To assess the performance and efficiency of PPO, DQN, and A2C, we utilize four primary metrics:

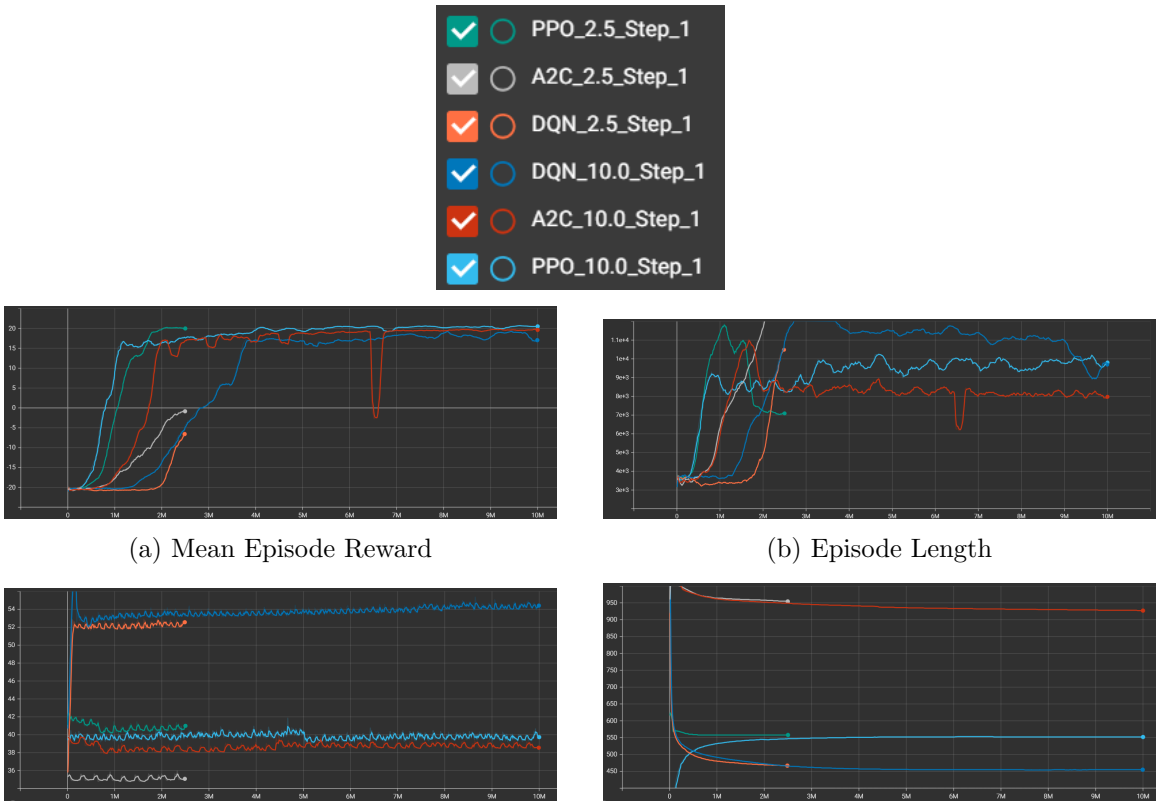
- **Mean Episode Reward ($\bar{R}$):** The average cumulative reward obtained by the agent over a rollout. In Pong, this ranges from -21 to +21. A score $> 0$ indicates the agent is winning more points than it loses.

- **Sample Efficiency:** Defined as the number of environment steps required to reach a specific performance threshold (e.g., $\bar{R} \geq 0$).

- **Wall-clock Training Time:** The total real-world time elapsed to complete the training duration (2.5M and 10M steps).

- **Resource Consumption:** Monitoring of system memory (RAM) and CPU utilization to determine the computational overhead of each algorithmic architecture.

## 4 Results




(a) Mean Episode Reward

(b) Episode Length

(c) Memory Usage

(d) Frames Per Second

Figure 1: Comparison of PPO, DQN, and A2C across multiple evaluation metrics.

### 4.1 Comparative Performance and Convergence

The training curves for all three models are illustrated in Figure 1. PPO exhibited superior sample efficiency, achieving a positive mean reward ($\bar{R} > 0$) at $\approx$ 1.0M steps, slightly outperforming A2C and significantly beats DQN. Both A2C and PPO converged to near-perfect scores ($\approx 20$) rapidly, with PPO maintaining the most stable plateau.

In contrast, DQN exhibited a significantly slower learning trajectory. Due to its high exploration requirements—as evidenced by the linear decay in its exploration rate—it did not achieve positive mean rewards until approximately 3.5 million steps. While DQN even-

tually reached an asymptotic performance comparable to the other models, its convergence rate is notably lower in this high-dimensional visual task.

### 4.2 Stability and Anomalies

A significant observation was recorded in the 10M step A2C trial. After reaching a plateau at 2M steps, the model suffered a sharp performance collapse at approximately 6.5M steps, with rewards dropping momentarily before recovering. This suggests potential instability in the policy gradient updates or a sensitivity to the learning rate scheduler in the late stages of training, a phenomenon not observed in the clipped objective updates of PPO.

### 4.3 Computational Efficiency

The trade-offs between algorithm complexity and resource usage are summarized in Table 1.

| **Metric** | **A2C** | **PPO** | **DQN** |
|---|---|---|---|
| Training Time (10M steps) | $\approx$ 3h 00m | $\approx$ 5h 00m | $\approx$ 6h 05m |
| Peak Memory Usage (MB) | Lowest | Moderate | Highest |
| Convergence (Steps to $> 0$) | $\approx$ 1.2M | $\approx$ 1.0M | $\approx$ 3.5M |
| Frames Per Second | $\approx$ 950 | $\approx$ 550 | $\approx$ 450 |

Table 1: Comparative metrics for A2C, PPO, and DQN on 10M steps.

DQN utilized substantially more memory than the other two models. This is attributed to the Experience Replay Buffer, which stores a large history of transitions to break temporal correlations. PPO and A2C, being on-policy (or near on-policy) methods, maintained a significantly smaller memory footprint. However, A2C proved to be the most time-efficient in terms of wall-clock duration, completing 10M steps in roughly half the time required by DQN, highlighting its lower computational overhead per update.

### 4.4 Scalability and Consistency: 2.5M vs. 10M Steps

Comparing the 2.5M step trials against the 10M step extended runs reveals whether algorithmic trends are consistent across different temporal scales.

- **Reinforcing Efficiency Trends:** The 2.5M step results strongly reinforce the conclusion that PPO and A2C are superior in terms of initial sample efficiency. In both scales, PPO and A2C achieved positive mean rewards and reached the score plateau within the first 1.5M steps. This suggests that for tasks with the complexity of Pong, 2.5M steps are sufficient to achieve near-optimal performance for policy-gradient methods.

- **The DQN Discrepancy:** The 2.5M step model for DQN appears to oppose the final success of the 10M model if viewed in isolation. At 2.5M steps, DQN is still in a state of net-negative reward ($\bar{R} \approx -6.57$), whereas PPO and A2C have already "solved" the environment. This highlights that DQN requires a longer "burn-in" period for exploration and buffer filling, meaning conclusions drawn solely from shorter 2.5M step runs would unfairly characterize DQN as a failing model.

- **A2C Stability Validation:** The 10M step run provides critical nuance that the 2.5M run misses. While both models show rapid convergence, only the 10M run reveals the catastrophic drop in performance for A2C at the 6.5M mark. This reinforces the conclusion that while A2C is highly efficient in the short term (as seen in the 2.5M run), it lacks the long-term objective stability provided by PPO's clipping mechanism.

Overall, the 2.5M models confirm the *efficiency* rankings of the algorithms, while the 10M models are necessary to draw conclusions regarding *asymptotic stability* and the eventual convergence of value-based methods like DQN.

## 5 Conclusion and Future Work

### 5.1 Conclusion

This study set out to compare three foundational deep reinforcement learning algorithms, PPO, DQN, and A2C, under identical conditions on the Atari Pong environment. By controlling for hardware, preprocessing, and vectorization, we were able to attribute performance differences directly to each algorithm's design choices. PPO proved to be the strongest overall performer. It reached a positive mean reward earliest at approximately 1.0 million steps, maintained the most stable training curve throughout, and did so with a moderate memory footprint. Its clipped surrogate objective effectively prevented destructive policy updates, which was most evident when compared against the collapse observed in A2C at 6.5 million steps. For practitioners working under resource constraints who need reliable, fast convergence, PPO is the clear recommendation. DQN presented a more nuanced result. At 2.5 million steps it was still in net-negative reward territory, which would suggest poor performance in isolation. However, the 10 million step run revealed that DQN continued improving long after PPO and A2C had plateaued, ultimately reaching competitive asymptotic performance. The primary limitation of DQN in this study was its experience replay buffer, which consumed significantly more memory than the other two algorithms and contributed to a much longer burn-in period before meaningful learning occurred. A2C demonstrated strong computational efficiency, completing 10 million steps in approximately 3 hours, roughly half the wall-clock time required by DQN. It also converged reasonably quickly, reaching positive rewards at around 1.2 million steps. However, the 10 million step run exposed a notable instability, with a sharp performance collapse at approximately 6.5 million steps before recovering. This suggests that A2C's policy gradient updates are more sensitive to learning rate scheduling in the later stages of training, a weakness that PPO's clipping mechanism avoids entirely. Overall, the results confirm that no single algorithm dominates across every dimension. PPO offers the best balance of speed, stability, and resource efficiency. DQN is best suited for scenarios where longer training is feasible and peak asymptotic performance is the priority. A2C is the most computationally lightweight option but requires additional tuning to ensure long-term stability.

### 5.2 Future Work

Several directions remain open for further investigation. First, hyperparameter tuning for all three models was kept relatively standard in this study to ensure a fair comparison. Future work could explore more aggressive tuning, such as adjusting learning rate schedules, entropy coefficients, and network architectures, to push each algorithm closer to its true performance ceiling. Second, this study evaluated models at fixed training budgets of 2.5 million and 10 million steps. A more granular checkpoint analysis, comparing all three algorithms at regular intervals throughout training, would provide clearer insight into exactly where and when one algorithm pulls ahead of another, which would be valuable for practitioners deciding how much compute to allocate. Third, the performance collapse observed in A2C at 6.5 million steps warrants further investigation. Exploring alternative learning rate schedulers or incorporating techniques such as gradient clipping could help determine whether this instability is inherent to A2C or simply a consequence of the specific hyperparameter configuration used here. Finally, extending this comparison beyond Pong to other Atari environments with different reward structures and action spaces would help determine how well these findings generalize. Pong is a relatively simple environment, and the relative rankings of these algorithms may shift considerably in more complex settings.

## References


Neil de la Fuente and Daniel A. Vidal Guerra. A comparative study of deep reinforcement learning models: Dqn vs ppo vs a2c. *arXiv preprint arXiv:2407.14151*, 2024. URL `https://arxiv.org/abs/2407.14151`.

Peter Henderson et al. Deep reinforcement learning that matters. *arXiv preprint arXiv:1709.06560*, 2019. URL `https://arxiv.org/abs/1709.06560`.

Michael Krebsbach. Deep reinforcement learning: Pong from pixels / playing pong with deep reinforcement learning, 2016. URL `https://karpathy.github.io/2016/05/31/rl/`.

Volodymyr Mnih et al. Playing atari with deep reinforcement learning. *arXiv preprint arXiv:1312.5602*, 2013. URL `https://arxiv.org/abs/1312.5602`.

Volodymyr Mnih et al. Asynchronous methods for deep reinforcement learning. *arXiv preprint arXiv:1602.01783*, 2016. URL `https://arxiv.org/abs/1602.01783`.

John Schulman et al. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017. URL `https://arxiv.org/abs/1707.06347`.

Mark Towers et al. Gymnasium: A standardized interface for reinforcement learning environments. *arXiv preprint arXiv:2407.17032*, 2025. URL `https://arxiv.org/abs/2407.17032`.